Problem Set B — Data and Simulation (Weeks –)

Programming and Numerical Methods for Economics (ECNM)
The University of Edinburgh · School of Economics

Instructions. This problem set covers material from Weeks and : pandas DataFram indexing, conditional subsetting, dummy variables, missing values, random number generation, distributions, Monte Carlo integration, AR( ) processes, and Markov chains.

This version includes solutions.

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(42)
```

Question — Building and exploring a DataFrame

The table below shows data on six European economies:

Country	GDP (bn €)	Population (m)	Unemployment (%)	Year
Germany		.	.	
France		.	.	
Italy		.	.	
Spain		.	.	
Netherlands		.	.	
Greece		.	.	

(a) Create a DataFrame `eu` from this data (use any method you prefer).

(b) Add a new column `gdp_pc` equal to GDP per capita (GDP / Population). Note the units.

(c) Select only the countries where unemployment is **above %**. Which countries appear?

(d) Select countries with GDP per capita **above the median** AND unemployment **below %**. only the `Country` and `gdp_pc` columns.

In [ ]:
```
# --- Solution ---

# (a)
data = {
```

```
    'Country': ['Germany', 'France', 'Italy', 'Spain', 'Netherlands', 'Gr
    'GDP': [3870, 2780, 2010, 1400, 1010, 220],
    'Population': [83.2, 67.8, 59.0, 47.4, 17.6, 10.4],
    'Unemployment': [3.1, 7.3, 7.6, 12.9, 3.5, 11.2],
    'Year': [2023, 2023, 2023, 2023, 2023, 2023]
}
eu = pd.DataFrame(data)
print(eu)

# (b)
eu['gdp_pc'] = eu['GDP'] / eu['Population']  # bn € per million people =
print("\nWith GDP per capita (thousands €):")
print(eu[['Country', 'gdp_pc']])

# (c)
high_unemp = eu.loc[eu['Unemployment'] > 7]
print("\nCountries with unemployment > 7%:")
print(high_unemp[['Country', 'Unemployment']])
# France, Italy, Spain, Greece

# (d)
median_gdppc = eu['gdp_pc'].median()
result = eu.loc[(eu['gdp_pc'] > median_gdppc) & (eu['Unemployment'] < 7),
                ['Country', 'gdp_pc']]
print(f"\nMedian GDP per capita: {median_gdppc:.2f} thousand €")
print("Above-median GDP/cap AND unemployment < 7%:")
print(result)
# Germany and Netherlands
```

Question — Dummies, missing values, and data cleaning

The DataFrame below records households in a survey:

```
survey = pd.DataFrame({
    'hh_id':    [1,    2,    3,     4,     5,     6,     7],
    'income':   [2500, 4100, np.nan, 1800, 3200,  np.nan, 5500],
    'region':   ['urban', 'rural', 'urban', 'rural', 'urban', 'rural',
'urban'],
    'children': [2,    0,    1,     np.nan, 3,     1,     np.nan]
})
```

(a) Create this DataFrame. How many missing values does each variable have?

(b) Drop all rows where `income` is missing. How many rows survive?

(c) On the surviving data, fill missing `children` with the **median** number of children (compute non-missing values).

(d) Create a dummy variable `urban` equal to if region is `'urban'`, otherwise.

(e) Compute the mean income separately for urban and rural households (use the cleaned data) group has a higher average?

In [ ]:
```
# --- Solution ---

# (a)
survey = pd.DataFrame({
    'hh_id':    [1,    2,    3,     4,     5,     6,     7],
    'income':   [2500, 4100, np.nan, 1800, 3200,  np.nan, 5500],
    'region':   ['urban', 'rural', 'urban', 'rural', 'urban', 'rural', 'u
    'children': [2,    0,    1,     np.nan, 3,     1,     np.nan]
})
print("Missing values per variable:")
print(survey.isnull().sum())

# (b)
clean = survey.dropna(subset=['income']).copy()
print(f"\nRows surviving after dropping missing income: {len(clean)}")

# (c)
med_children = clean['children'].median()
print(f"Median children (non-missing): {med_children}")
clean['children'] = clean['children'].fillna(med_children)
print(clean[['hh_id', 'children']])

# (d)
clean['urban'] = 1 * (clean['region'] == 'urban')
print("\nWith urban dummy:")
print(clean[['hh_id', 'region', 'urban']])

# (e)
mean_by_region = clean.groupby('region')['income'].mean()
print("\nMean income by region:")
print(mean_by_region)
# Urban households have higher average income.
```

Question — Random numbers and distributions

(a) Set the seed to . Draw , observations from a Normal distributi
$\mu = $ and $\sigma = $. Store them in `x`.

(b) Draw , observations from a **log-normal** distribution by computing `z = np.exp(x)`. Plot histograms of both `x` and `z` side by side (use `plt.subplots(1, 2)`). T
panel.

(c) Create your own discrete distribution with support `[0, 1, 2, 3]` and probabilities `[0.5, 0.2, 0.1]`. Draw , samples from it. Verify empirically that the sample mean to the theoretical mean $E[X] = \sum_i x_i \, p_i$.

In [ ]:
```
# --- Solution ---

# (a)
np.random.seed(2024)
x = np.random.normal(loc=5, scale=2, size=10000)
print(f"Sample mean of x: {np.mean(x):.4f} (should be close to 5)")
print(f"Sample std of x:  {np.std(x):.4f} (should be close to 2)")
```

```
# (b)
z = np.exp(x)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(x, bins=60, density=True, alpha=0.7, color='steelblue')
axes[0].set_title('Normal distribution (mu=5, sigma=2)')
axes[0].set_xlabel('x')
axes[0].set_ylabel('Density')

axes[1].hist(z, bins=100, density=True, alpha=0.7, color='coral')
axes[1].set_title('Log-normal distribution')
axes[1].set_xlabel('z = exp(x)')
axes[1].set_ylabel('Density')
axes[1].set_xlim(0, np.percentile(z, 99))  # trim extreme tail for visibi

plt.tight_layout()
plt.show()

# (c)
support = [0, 1, 2, 3]
probs   = [0.5, 0.2, 0.2, 0.1]

draws = np.random.choice(support, size=10000, p=probs)

theoretical_mean = sum(x_i * p_i for x_i, p_i in zip(support, probs))
sample_mean = np.mean(draws)

print(f"Theoretical E[X] = {theoretical_mean}")
print(f"Sample mean      = {sample_mean:.4f}")
```

Question — Monte Carlo integration

Suppose $X \sim N(,)$ and we want to compute $E[g(X)]$ where $g(x) = e^{-x^ / } \cdc

(a) Write a function `g(x)` implementing the expression above.

(b) Estimate $E[g(X)]$ by Monte Carlo with $N = {,} {,} $ draws. Print the res

(c) Investigate convergence: compute the Monte Carlo estimate for $N = , {,} , {,} , {,} {,} $. Store the estimates in a list them against N (use a log scale on the x-axis). Does the estimate stabilise?

In [ ]:
```
# --- Solution ---

# (a)
def g(x):
    return np.exp(-x**2 / 2) * x**2

# (b)
np.random.seed(2024)
X = np.random.normal(0, 1, 1_000_000)
mc_estimate = np.mean(g(X))
```

```
print(f"Monte Carlo estimate of E[g(X)] with N=1,000,000: {mc_estimate:.6

# (c)
N_values = [100, 1000, 10_000, 100_000, 1_000_000]
estimates = []

np.random.seed(2024)
for N in N_values:
    draws = np.random.normal(0, 1, N)
    estimates.append(np.mean(g(draws)))

fig, ax = plt.subplots(figsize=(8, 4))
ax.semilogx(N_values, estimates, 'o-', color='steelblue', markersize=8)
ax.axhline(mc_estimate, color='red', linestyle='--', alpha=0.5, label=f'N
ax.set_xlabel('Number of draws (N)')
ax.set_ylabel('E[g(X)] estimate')
ax.set_title('Monte Carlo convergence')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# The estimate stabilises as N grows — the law of large numbers at work.
```

Question — AR( ) processes

Consider the AR( ) process: $y_{t+ } = a + \rho \, y_t + \varepsilon_t$, where $\varepsilon_t '
N(, \sigma^)$.

(a) Write a function `simulate_ar1(T, rho, a=0, sigma=1, y0=0)` that returns a NumP
of length T containing the simulated path.

(b) Set `rho=0.9` , `a=1` , `sigma=0.5` , `T=200` . Simulate and plot the process. Add a horizor
at the **unconditional mean** $\mu^ = a/(-\rho)$. Does the process fluctuate around this v

(c) Now set `rho=1.01` (keeping the other parameters). Simulate periods and plo
happens? Is this process stationary? Explain in one sentence.

In [ ]:
```
# --- Solution ---

# (a)
def simulate_ar1(T, rho, a=0, sigma=1, y0=0):
    y = np.empty(T)
    y[0] = y0
    for t in range(1, T):
        eps = np.random.normal(0, sigma)
        y[t] = a + rho * y[t-1] + eps
    return y

# (b)
np.random.seed(2024)
rho, a, sigma, T = 0.9, 1, 0.5, 200
y_stat = simulate_ar1(T, rho, a, sigma)
mu_star = a / (1 - rho)
```

```
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(y_stat, color='steelblue', linewidth=0.8)
ax.axhline(mu_star, color='red', linestyle='--', label=f'$\\mu^* = {mu_st
ax.set_xlabel('t')
ax.set_ylabel('$y_t$')
ax.set_title(f'AR(1) with $\\rho={rho}$, $a={a}$ (stationary)')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Sample mean: {np.mean(y_stat):.2f}, Unconditional mean: {mu_star:
# Yes, the process fluctuates around mu*.

# (c)
np.random.seed(2024)
y_nonstat = simulate_ar1(200, rho=1.01, a=1, sigma=0.5)

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(y_nonstat, color='coral', linewidth=0.8)
ax.set_xlabel('t')
ax.set_ylabel('$y_t$')
ax.set_title('AR(1) with $\\rho=1.01$ (non-stationary)')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# The process explodes — it is non-stationary because |rho| > 1,
# so shocks accumulate rather than decay.
```

Question — Markov chains

An economy has three states of the business cycle: **Recession**, **Normal**, and **Boom**. The quarte transition matrix is:

$$ P = \begin{pmatrix} . & . & . \\ . & . & . \\ . & . & . \end{pmatrix} $$

where rows are current states (Recession, Normal, Boom) and columns are next-period states.

(a) Use `quantecon.MarkovChain` to create this Markov chain with state labels `['Recessi 'Normal', 'Boom']`.

(b) Simulate , periods starting from `'Normal'`. Compute the fraction of time each state.

(c) Compute the **stationary distribution** using `mc.stationary_distributions`. Compare your simulation fractions from (b).

(d) Suppose GDP growth in each state is: Recession = − %, Normal = %, Boom = % the stationary distribution, what is the **long-run average** GDP growth rate for this economy?

In [ ]:

```python
# --- Solution ---

import quantecon as qe

# (a)
P = [[0.70, 0.25, 0.05],
     [0.10, 0.80, 0.10],
     [0.05, 0.30, 0.65]]

mc = qe.MarkovChain(P, state_values=['Recession', 'Normal', 'Boom'])

# (b)
np.random.seed(2024)
sim = mc.simulate(ts_length=10000, init='Normal')

print("Simulation fractions (10,000 periods):")
for state in ['Recession', 'Normal', 'Boom']:
    frac = np.mean(sim == state)
    print(f"  {state}: {frac:.4f}")

# (c)
psi_star = mc.stationary_distributions[0]
print("\nStationary distribution:")
for state, prob in zip(['Recession', 'Normal', 'Boom'], psi_star):
    print(f"  {state}: {prob:.4f}")
# The simulation fractions closely match the stationary distribution.

# (d)
gdp_growth = np.array([-1, 2, 4])  # percent
long_run_avg = psi_star @ gdp_growth
print(f"\nLong-run average GDP growth: {long_run_avg:.2f}%")
```

End of Problem Set B.